



Ocar

Intercity Cab Platform · Odisha

Infrastructure & Delivery Pipeline Handover

A complete operational reference for the Ocar production platform: how code reaches production, how traffic is distributed, how capacity is added and removed, and the exact procedures for every routine and emergency operation. Written for an engineer taking independent operational control of the AWS account and GitHub repository.

DOCUMENT

Infrastructure & Delivery Handover

REVISION

1.0 — 20 August 2026

REPOSITORY

cardevelopment01-tech/0car-monorepo

AWS ACCOUNT / REGION

822127610728 · ap-south-1

SOURCE OF TRUTH

Verified against `main` @ 4a07a95

CLASSIFICATION

Confidential — client engineering

CONTENTS

- | | | | |
|---|--|----|-------------------------------------|
| 1 | Scope, method and how to read this | 10 | Changing instance type and size |
| 2 | System architecture | 11 | Configuration and secret management |
| 3 | Environment and state isolation | 12 | Data layer: Postgres and Valkey |
| 4 | Access model and credentials | 13 | Observability |
| 5 | Continuous integration | 14 | Operational procedures |
| 6 | Continuous delivery | 15 | Failure modes and recovery |
| 7 | Deployment strategy and its alternatives | 16 | Load testing |
| 8 | Request distribution and routing rules | 17 | Known gaps and risk register |
| 9 | Capacity: scaling out and in | 18 | Appendices |

SECTION 01

Scope, method and how to read this

This document describes the system as it is actually configured today, not as it was designed or intended. Every value, filename, job name and command was read directly out of the repository at the commit named on the cover page.

1.1 What this covers

The delivery pipeline (CI and CD), the AWS runtime environment, and the operational procedures for running it. It deliberately does not cover application-level product behaviour — ride matching, fare calculation, payment reconciliation — which is documented separately in `CLAUDE.md` and the module design specs under `docs/superpowers/specs/`.

1.2 Method and verifiability

Nothing here is asserted from memory or convention. Each factual claim is traceable to a file in the repository, and the relevant file path is named alongside it so any statement can be checked in under a minute. Where behaviour depends on an AWS semantic rather than on our code — for example what `MinHealthyPercentage = 100` actually causes the Auto Scaling service to do — the claim was verified against AWS's own current documentation rather than inferred.

Section 17 lists what is *not* right. It is deliberately placed in the same document rather than omitted, because a handover that only describes the working parts is not a handover. Several items in it are consequential.

1.3 Reading paths

IF YOU WANT TO...	READ
Understand the shape of the system	Sections 2, 3, 8
Ship a change and watch it land	Sections 5, 6, 7
Add or remove capacity	Sections 9, 10
Perform a specific operation	Section 14 (procedures <code>OPS-01</code> ... <code>OPS-12</code>)
Respond to an incident	Section 15, then Section 14
Assess risk before signing off	Section 17

1.4 Conventions

<code>Live</code>	Implemented, deployed, and verified working in production.
<code>Partial</code>	Implemented in code but not fully rolled out, or working with a known limitation.
<code>Gap</code>	Not implemented, or implemented incorrectly. Tracked in Section 17.
<code>N/A</code>	Deliberately out of scope at current scale, with reasoning given.

Commands shown as `$` -free single lines are safe to copy directly. Any command that mutates production state is marked in its procedure block with an explicit blast-radius note. Placeholders appear in angle brackets: `<ASG_NAME>` .

SECTION 02

System architecture

A single-region, load-balanced Node.js fleet on EC2, fronted by an Application Load Balancer, backed by managed Postgres and Valkey inside the same VPC. There is no container orchestrator and no service mesh; this is deliberate and is explained in §2.4.

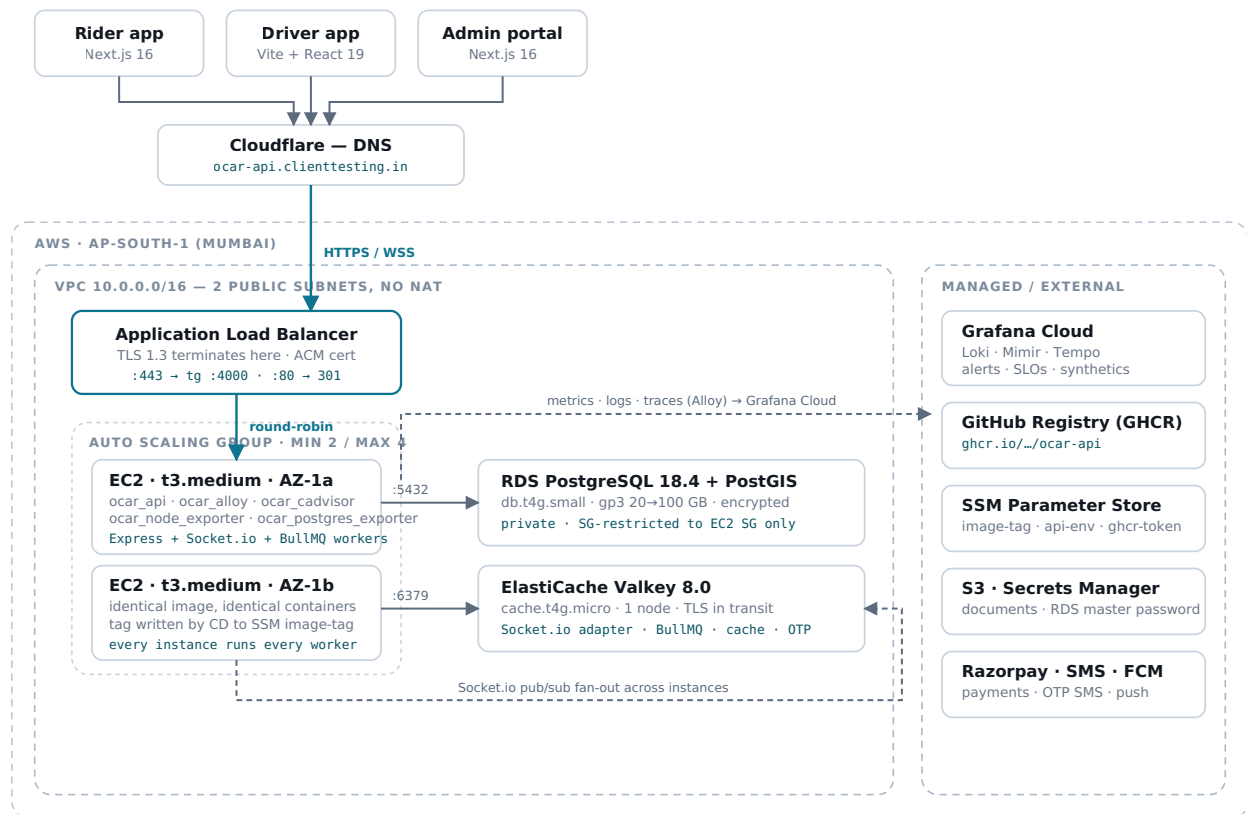


Figure 1 — Production topology. Solid teal marks the live request path. Every EC2 instance is identical and stateless at the process level: it terminates WebSockets, serves HTTP, and runs the full set of BullMQ background workers. Cross-instance coordination happens exclusively through Valkey — the Socket.io Redis adapter for real-time fan-out, and BullMQ for queued work.

2.1 Component inventory

COMPONENT	CONFIGURATION	DEFINED IN
Load balancer	ALB, internet-facing, 2 AZs, <code>drop_invalid_header_fields</code> on	<code>infra/terraform/alb.tf</code>
TLS	ACM cert, DNS-validated; policy <code>ELBSecurityPolicy-TLS13-1-2-Res-PQ-2025-09</code>	<code>acm.tf</code> , <code>alb.tf</code>
Compute	ASG, <code>min 2 / desired 2 / max 4</code> , <code>t3.medium</code> , Ubuntu 24.04	<code>asg.tf</code> , <code>launch-template.tf</code>
Database	RDS Postgres 18.4, <code>db.t4g.small</code> , single-AZ, 7-day backups	<code>rds.tf</code>
Cache / queue / pub-sub	ElastiCache Valkey 8.0, one node, <code>maxmemory-policy=noeviction</code>	<code>elasticache.tf</code>
Network	One VPC, two public subnets, IGW, no NAT, no private subnets	<code>vpc.tf</code>
Isolation	Security-group chaining — see §2.3	<code>security-groups.tf</code> , <code>rds.tf</code> , <code>elasticache.tf</code>
Image registry	GHCR, private, pulled with a PAT held in SSM	<code>deploy.yml</code> , <code>user_data.sh.tpl</code>
Observability	One Grafana Alloy agent per instance → Grafana Cloud	<code>infra/alloy/config.alloy</code>

2.2 What runs on an instance

Each instance runs five containers, started by Docker Compose from the boot script. There is no process manager, no systemd unit for the app, and no SSH-based deployment — an instance is disposable and rebuilds itself entirely from SSM at boot.

CONTAINER	IMAGE	ROLE AND LIMITS
ocar_api	ghcr.io/.../ocar-api	Express + Socket.io + all BullMQ workers. <code>mem_limit 2g</code> , <code>mem_reservation 512m</code> , <code>cpus 1.5</code> .
ocar_alloy	grafana/alloy:v1.18.1	Ships logs, metrics and traces to Grafana Cloud. Holds the only copy of the Grafana credentials.
ocar_node_exporter	prom/node-exporter:v1.9.1	Host-level CPU, memory, disk, network.
ocar_cadvisor	ghcr.io/google/cadvisor:v0.60.5	Per-container CPU and memory. <code>mem_limit 200m</code> , <code>--docker_only</code> , 30 s housekeeping.
ocar_postgres_exporter	postgres-exporter:v0.20.1	Postgres internals including <code>pg_stat_statements</code> .

NOTE

`docker-compose.prod.yml` also defines an `nginx` service. It is **not started** on ASG instances — the boot script names services explicitly (`up -d api alloy node_exporter postgres_exporter cadvisor`) and omits it, because the ALB terminates TLS. The service definition is retained only for the legacy single-box deployment.

2.3 Isolation model

There are no private subnets, so isolation is enforced by security groups rather than by network placement. This is a deliberate trade — a NAT gateway costs roughly \$32–45/month plus data processing charges, which is not justified at current scale. The security boundary is equivalent provided the groups are correct, and they are chained by group ID rather than by CIDR:

GROUP	INBOUND	EFFECT
ocar-alb-sg	:80, :443 from <code>0.0.0.0/0</code>	The only internet-reachable surface.
ocar-ec2-sg	:4000 from <code>ocar-alb-sg</code> only	Instances are unreachable from the internet despite holding public IPs.
ocar-<env>-rds-sg	:5432 from <code>ocar-ec2-sg</code> only	Database unreachable from anywhere but the fleet. <code>publicly_accessible = false</code> .
ocar-<env>-valkey-sg	:6379 from <code>ocar-ec2-sg</code> only	Cache unreachable from anywhere but the fleet.

SSH is closed by default. `var.ssh_debug_cidr` defaults to `null` , which removes the rule entirely rather than leaving it open to a placeholder range. Administrative shell access, when required, is via SSM Session Manager — see §4.4.

2.4 Architectural decisions worth stating explicitly

DECISION	REASONING
Modular monolith, not microservices	One deployable unit with clean internal module boundaries (<code>api/src/modules/</code> *). At 1–2k rides/day with a team of this size, service decomposition adds network failure modes, distributed transactions and deployment coordination without removing any current constraint. The boundaries are already drawn in code, so extraction later is refactoring, not a rewrite.
EC2 + Compose, not ECS or Kubernetes	The fleet is two to four identical instances running one application. An orchestrator's value is bin-packing heterogeneous workloads and managing large-scale scheduling, neither of which applies. ASG instance refresh already provides rolling, health-gated replacement.
Raw SQL, no ORM	The platform is geospatially heavy (PostGIS <code>ST_MakePoint</code> / <code>ST_MakeLine</code> casts throughout) and uses append-only versioning patterns for rate cards that most ORMs default against. Documented at length in <code>docs/architecture/2026-08-04-production-deployment-discussion-notes.md</code> .
Single region	The user base is Odisha. Multi-region adds replication lag, split-brain risk and roughly double the infrastructure cost for no latency benefit to the actual audience.
Workers co-located with the API	Every instance runs every BullMQ worker. BullMQ's Redis-backed locking makes this safe. Splitting workers onto their own ASG is the correct move once queue load and request load need to scale independently; today they do not.

SECTION 03

Environment and state isolation

Two environments share one Terraform configuration and are separated by explicit backend selection. There is no `terraform workspace` anywhere in this repository — that was a deliberate choice, because workspaces switch state invisibly.

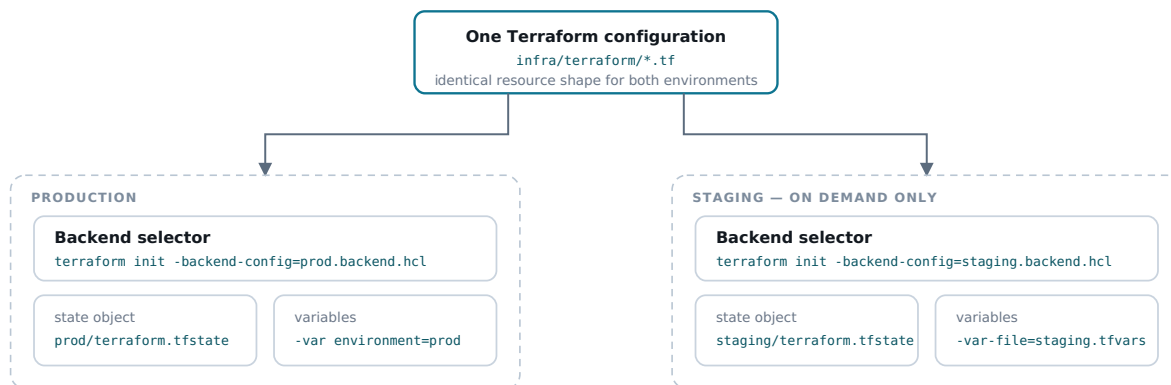


Figure 2 — State isolation. The `backend "s3" {}` block in `providers.tf` is intentionally empty. A bare `terraform init` with no flags therefore fails rather than silently defaulting to production — which is exactly what it used to do before staging existed.

3.1 Environment matrix

PROPERTY	PRODUCTION	STAGING
Lifecycle	Permanent	Created on demand for load tests, destroyed after
State key	<code>prod/terraform.tfstate</code>	<code>staging/terraform.tfstate</code>
Backend file	<code>prod.backend.hcl</code>	<code>staging.backend.hcl</code>
Variables	<code>-var="environment=prod"</code>	<code>-var-file=staging.tfvars</code>
Domain	<code>ocar-api.clienttesting.in</code>	<code>staging.ocar-api.clienttesting.in</code>
Instance type	<code>t3.medium</code>	<code>t3.medium</code> — identical, by design
ASG sizing	2 / 2 / 4	2 / 2 / 4 — see gap G-07
Deletion protection	Enabled on RDS	Disabled; <code>skip_final_snapshot = true</code>
Applied by	Manual <code>terraform apply</code> from a workstation	<code>staging-infra.yml</code> workflow dispatch, or locally

Staging intentionally matches production's infrastructure shape. The entire purpose of the environment is to produce load-test numbers that can be trusted, which is impossible if it runs on smaller instances or a different cache tier. `staging.tfvars` therefore overrides only two values — `environment` and `domain_name` — and leaves everything else at the shared default.

3.2 Shared, account-wide resources

Three things are deliberately *not* owned by either environment's state, because they are account-level singletons that both environments read:

RESOURCE	OWNED BY	CONSUMED AS
Terraform state bucket + KMS key	<code>infra/terraform/bootstrap/</code>	<code>data.aws_s3_bucket</code> , <code>data.aws_kms_key</code>
GitHub Actions OIDC provider	<code>infra/terraform/bootstrap/</code>	<code>data.aws_iam_openid_connect_provider</code>
Grafana Cloud alerts, SLOs, synthetics	<code>infra/terraform/observability/</code>	Own state at <code>observability/terraform.tfstate</code>

The bootstrap module is applied once and then effectively never touched. Both prod and staging reference its outputs through `data` sources only, so neither can create, modify or destroy them. State locking uses S3's native `use_lockfile` rather than the older DynamoDB table pattern, so there is no separate lock table to maintain.

SECTION 04

Access model and credentials

Nothing in the pipeline authenticates to AWS with a long-lived access key. All three GitHub Actions roles are assumed through OIDC federation, scoped by the workflow's own signed identity claim, and expire with the job.

4.1 The three CI roles

Defined in `infra/terraform/github-oidc.tf`. Each is trusted only for a specific GitHub OIDC subject claim, so a workflow cannot assume a role it was not designed for even if someone edits the workflow file.

ROLE	TRUSTED SUBJECT	PERMITTED ACTIONS
ocar-prod-gha-deploy	repo:...:environment:production	<code>ssm:GetParameter / PutParameter</code> on the <code>image-tag</code> parameter only; <code>autoscaling:StartInstanceRefresh / CancelInstanceRefresh</code> on this ASG only; <code>DescribeInstanceRefreshes</code> ; <code>ec2:DescribeInstances</code> ; <code>ssm:SendCommand</code> + <code>GetCommandInvocation</code> .
ocar-prod-gha-plan	repo:...:pull_request	Read-only across the services this config manages, plus read/write on the <code>prod/*</code> state prefix and the state KMS key. Never runs on <code>main</code> .
ocar-prod-gha-staging	repo:...:environment:staging	Full apply/destroy, but only on the <code>staging/*</code> state prefix, <code>/ocar/staging/*</code> parameters, and <code>ocar-staging-*</code> IAM roles.

DETAIL WORTH KNOWING

The deploy role's trust policy matches on `environment:production` , not on `ref:refs/heads/main` . This is not arbitrary — when a job declares `environment:` , GitHub changes the OIDC subject claim format. The ref-based condition was tried first, failed, and the actual rejected claim was read out of CloudTrail to determine the correct form. If you add a new deploy job, it must declare the same environment or the assume-role will fail.

4.2 The EC2 instance role

Instances assume `ocar-<env>-ec2-role` (`infra/terraform/iam.tf`). It grants `ssm:GetParameter` on exactly six named parameters and `kms:Decrypt` on the `alias/aws/ssm` key — nothing broader:

- `/ocar/<env>/image-tag` — which image version to run
- `/ocar/<env>/ghcr-token` — GHCR pull credential (`SecureString`)
- `/ocar/<env>/api-env` — the application's entire environment file (`SecureString`)
- `/ocar/<env>/migration-database-url` — direct DB URL for migrations (`SecureString`)
- `/ocar/<env>/docker-compose-prod` — Compose file, Terraform-managed
- `/ocar/<env>/alloy-config` — Alloy config, Terraform-managed

The AWS-managed `AmazonSSMManagedInstanceCore` policy is also attached. Its inline comment still describes it as temporary debugging access, which is now **out of date** — the deploy pipeline routes database migrations to a live instance through `ssm:SendCommand` , and that requires the instance to be a registered SSM managed instance. Removing this attachment would break every deploy. See gap G-04 .

4.3 What you are receiving

CREDENTIAL	SCOPE	NOTES
GitHub repository access	Per contract	Grants ability to read all workflow definitions, run workflows, and read Actions logs.
AWS console / CLI access	Per contract	Region <code>ap-south-1</code> . Verify your effective permissions with <code>aws sts get-caller-identity</code> and <code>aws iam simulate-principal-policy</code> before assuming a capability.

OPERATIONAL SAFETY

Three destructive capabilities exist and have no confirmation gate today. Please read gaps `G-05` and `G-06` before using them: running `staging-infra.yml` with `action: destroy` executes `terraform destroy -auto-approve` against real infrastructure with no approval step; a push directly to `main` bypasses review entirely; and a merge to `main` that touches `api/**` deploys to production automatically with no human pause.

4.4 Getting a shell on an instance

SSH is not open. Use SSM Session Manager, which is authenticated by IAM and audited in CloudTrail:

```
# list instances belonging to the ASG
aws ec2 describe-instances \
  --filters "Name=tag:aws:autoscaling:groupName,Values=<ASG_NAME>" \
    "Name=instance-state-name,Values=running" \
  --query 'Reservations[].Instances[].[InstanceId,PrivateIpAddress,Placement.AvailabilityZone]' \
  --output table

# open a shell
aws ssm start-session --target <INSTANCE_ID>

# once inside – the app lives here
cd /opt/ocar
sudo docker compose -f docker-compose.prod.yml ps
sudo docker logs --tail 200 -f ocar_api
sudo cat /var/log/user-data.log # full boot transcript
```

NOTE

`/var/log/user-data.log` is the single most useful file when an instance fails to become healthy. The boot script runs under `set -x`, so it contains a complete trace of every step — with credential assignments explicitly untraced so tokens never land in the log.

SECTION 05

Continuous integration

`.github/workflows/ci.yml`. Seven independent jobs, no dependency edges between them, all starting simultaneously. Triggered on every push to every branch and on every pull request targeting `main`.

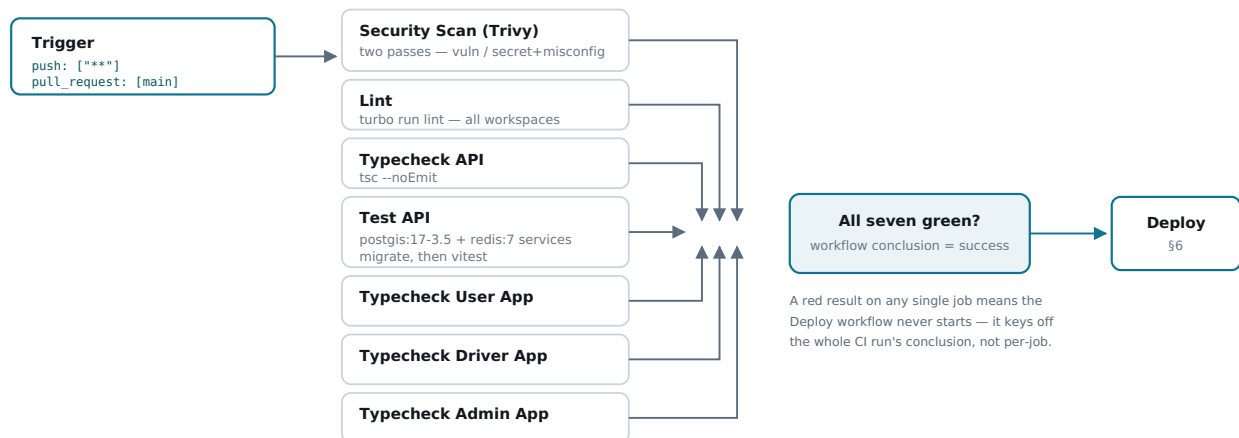


Figure 3 — CI job graph. The seven jobs are fully parallel. Deploy is chained off the overall run conclusion via `workflow_run`, which is what makes a single failing check structurally block deployment rather than merely discouraging it.

5.1 Job reference

JOB (DISPLAY NAME)	WHAT IT DOES	FAILS WHEN
Security Scan (Trivy)	Two separate filesystem scans of the whole repo.	Pass 1 — dependency CVEs at CRITICAL only. Pass 2 — secrets and IaC misconfiguration at CRITICAL and HIGH .
Lint	<code>pnpm turbo run lint</code> across every workspace.	Any lint error in any package.
Typecheck API	<code>tsc --noEmit</code> on <code>@ocar/api</code> .	Any type error. Note <code>exactOptionalPropertyTypes</code> is on, so this is stricter than default.
Test API	Boots real <code>postgres/postgis:17-3.5</code> and <code>redis:7-alpine</code> service containers, writes a test <code>.env</code> , runs the full migration chain, then <code>vitest</code> .	A migration fails to apply, or any test fails.
Typecheck User / Driver / Admin App	Three separate <code>tsc --noEmit</code> jobs, one per frontend.	Any type error in that app.

5.2 Design points

CHOICE	WHY
Trivy pinned to a commit SHA	<code>aquasecurity/trivy-action@ed142fd0...</code> rather than <code>@v0.36.0</code> . Version tags on that action were force-pushed during a March 2026 supply-chain incident; a mutable tag is not a trustworthy reference for a security-scanning step.
Dependency scan is <code>CRITICAL</code> -only	The monorepo carries roughly 27 pre-existing <code>HIGH</code> transitive findings, mostly DoS-class and largely in frontend packages. Gating on them would block every pull request indefinitely and train the team to ignore the scanner. Secrets and misconfiguration — where a finding is nearly always real and actionable — stay at <code>HIGH</code> . This is a conscious, revisitable trade-off, recorded as <code>G-15</code> .
Migrations run in CI against a real Postgres	A migration that fails to apply cleanly fails the build before it can reach production. This is the only place the full migration chain is exercised from empty on every change.
Per-job <code>concurrency</code> with <code>cancel-in-progress</code>	Pushing twice in quick succession cancels the superseded run per job, saving runner minutes without cancelling unrelated branches.
Node 22, pnpm 9, frozen lockfile	<code>--frozen-lockfile</code> means CI fails rather than silently resolving a different dependency tree than the one committed.

KNOWN WEAKNESS

Test coverage is thin. Five test files exist in the API — four covering ride assignment and broadcast, one covering maintenance middleware — and there are none for payments, authentication, OTP, wallet or settlement. `vitest` and `@vitest/coverage-v8` are installed and a `test:coverage` script exists, but no coverage threshold is enforced in CI. Recorded as `G-13`. The *pipeline* is production-grade; the *test suite it runs* is not yet.

SECTION 06

Continuous delivery

`.github/workflows/deploy.yml`. Three jobs — `changes`, `build-push`, `deploy` — chained by `needs:`. Triggered by `workflow_run` when the CI workflow completes on `main`, and gated on that run having concluded successfully.

```

graph TD
    subgraph "GITHUB ACTIONS"
        direction TB
        A01["01 Detect changes  
diff vs last deployed SHA"] --> A02["02 Build & push image  
tag: sha-<short>"]
        A02 --> A03["03 Assume AWS role  
OIDC, no static keys"]
    end

    subgraph "AWS CONTROL PLANE"
        direction TB
        A04["04 Locate a running ASG instance  
ec2:DescribeInstances - filter by ASG tag"] --> A06["06 Record current tag, then write new tag  
/ocar/prod/image-tag ← rollback anchor"]
        A06 --> A07["07 Start instance refresh  
MinHealthy 100 / MaxHealthy 100 → one at a time"]
        A07 --> A09["09 Poll refresh status  
60 attempts × 30 s – 30 min ceiling"]
        A09 --> R01["Roll back  
restore prior tag + refresh"]
    end

    subgraph "RUNNING FLEET"
        direction TB
        A05["05 Run migrations on that instance  
ssm:SendCommand → docker run --name db_migrate  
old code still serving traffic throughout"] --> G01["Migration succeeded? — gate"]
        G01 --> A06
        A08["08 Rolling replacement — see Figure 5  
launch → healthy → drain → terminate → repeat"] --> A07
        A10["10 ALB health check  
GET /health – 8 attempts × 5 s"] --> A11["11 Smoke test a real DB-backed route  
GET /api/v1/geo/cities – body must contain \"slug\""]
        A11 --> D01["Deploy complete"]
    end

    A03 --> A04
    A04 --> A05
    A05 --> G01
    G01 --> A06
    A06 --> A07
    A07 --> A08
    A08 --> A09
    A09 --> R01
    R01 --> A04
    A09 -.-> A10
    A10 -.-> A11
    A11 -.-> D01
  
```

The diagram illustrates a multi-step deployment strategy for AWS Lambda functions, organized into three main phases:

- GITHUB ACTIONS:**
 - 01 Detect changes:** diff vs last deployed SHA.
 - 02 Build & push image:** tag: sha-<short>
 - 03 Assume AWS role:** OIDC, no static keys.
- AWS CONTROL PLANE:**
 - 04 Locate a running ASG instance:** ec2:DescribeInstances - filter by ASG tag.
 - 06 Record current tag, then write new tag:** /ocar/prod/image-tag ← rollback anchor.
 - 07 Start instance refresh:** MinHealthy 100 / MaxHealthy 100 → one at a time.
 - 09 Poll refresh status:** 60 attempts × 30 s – 30 min ceiling.
 - Roll back:** restore prior tag + refresh.
- RUNNING FLEET:**
 - 05 Run migrations on that instance:** ssm:SendCommand → docker run --name db_migrate old code still serving traffic throughout.
 - Migration succeeded? — gate:** Decision point for migration success.
 - 08 Rolling replacement — see Figure 5:** launch → healthy → drain → terminate → repeat.
 - 10 ALB health check:** GET /health – 8 attempts × 5 s.
 - 11 Smoke test a real DB-backed route:** GET /api/v1/geo/cities – body must contain "slug".
 - Deploy complete:** Final state after successful deployment.

rebuilt the image and cycled the entire ASG for nothing. Neither path is ever copied into the image — `api/Dockerfile` only `COPY`s the lock files, the root `package.json`, and `api/`. Second, diffing against `HEAD^` instead of the last deployed SHA silently skipped five commits when someone pushed six commits directly to `main` and only the last was documentation.

Step 2 — Build and push

Multi-stage build from `api/Dockerfile`. Four stages: install all dependencies, compile TypeScript, produce a symlink-free production `node_modules` via `pnpm deploy --prod`, then assemble a minimal `node:22.16-alpine` runtime image. The final image runs as the non-root `node` user and carries its own `HEALTHCHECK`.

Images are tagged `sha-<short-sha>`, plus `latest` on the default branch. The immutable SHA tag is what the deployment actually references — `latest` exists for convenience only and is never what an instance boots from.

Steps 4–5 — Migrations, and why they run through SSM

RDS is not publicly reachable; only the EC2 security group can open port 5432. A GitHub-hosted runner therefore cannot connect to the database directly. Rather than punch a hole in the security group or introduce a bastion, the workflow finds a live ASG instance and asks SSM to run the migration container *there*:

```
docker run --rm --name db_migrate \
  -e MIGRATION_DATABASE_URL="$DB_URL" \
  --entrypoint node ghcr.io/<owner>/ocar-api:<tag> dist/db/migrate.js
```

Three properties of this arrangement are worth understanding:

- **No secret crosses the runner.** The instance reads the database URL from SSM itself using its own IAM role. The command text sent by the workflow contains a parameter `name`, never a value.
- **Migrations run against the new image but before the rollout.** The old application version is still serving every request while this happens, which means migrations must be backward-compatible with the currently-running code. Additive changes are safe; dropping or renaming a column that live code still reads is not.
- **`--name db_migrate` is load-bearing.** Without an explicit name, Docker assigns a random one, cAdvisor discovers it as a new container, and every deploy permanently added active series to the Grafana Cloud metrics tenant. That caused a real cardinality incident on 19 August 2026. Alloy's relabel rules now keep only containers matching `ocar_*`, so this container is filtered out by name.

IDEMPOTENCY

`dist/db/migrate.js` also runs on every container start via `api/entrypoint.sh`. So on a normal deploy the migration runner executes several times — once from this step, then once per replacement instance. This is safe because the runner skips already-applied files and serialises concurrent invocations with a `pg_advisory_lock`. The explicit step exists purely to *gate* the rollout on migration success rather than discovering a failure one instance at a time.

Steps 6–7 — Tag swap and refresh

The current value of `/ocar/prod/image-tag` is captured first — this is the rollback anchor, and capturing it before overwriting is the entire reason automatic rollback is possible. The new tag is then written, and an instance refresh is started. Note that instances read the tag **only at boot**; writing the parameter changes nothing until instances are replaced.

Steps 9–11 — Verification

CHECK	BUDGET	PROVES
Refresh status poll	$60 \times 30 \text{ s} \approx 30 \text{ min}$	AWS considers every replacement instance healthy in the target group.
<code>GET /health</code>	$8 \times 5 \text{ s}$	The ALB is routing and an instance answers.
<code>GET /api/v1/geo/cities</code>	$3 \times 3 \text{ s}$	A real database-backed route returns real rows — the response body must contain <code>"slug"</code> .

Both HTTP checks are issued against the ALB's DNS name with `curl -k` and an explicit `Host: ocar-api.clienttesting.in` header. The certificate is issued for the domain, not for the ALB hostname, so verification is skipped deliberately at this step; the header is what makes the request route correctly.

The smoke test matters more than the health check. As documented in gap [G-01](#), `/health` currently returns HTTP 200 even when the database is unreachable — so the smoke test is the only step in the pipeline that genuinely proves the database path works end to end.

Step 12 — Rollback

Any failure at steps 9, 10 or 11 restores the previously recorded tag and starts a second instance refresh, then exits non-zero. Two limits apply:

- **Migrations are not reverted.** Schema changes are forward-only. If the migration itself is the fault, it must be fixed with a new migration, not undone.
- **Rollback depth is one.** Only the immediately preceding tag is recorded. Going back further is a manual operation — procedure [OPS-02](#).

Deployment strategy and its alternatives

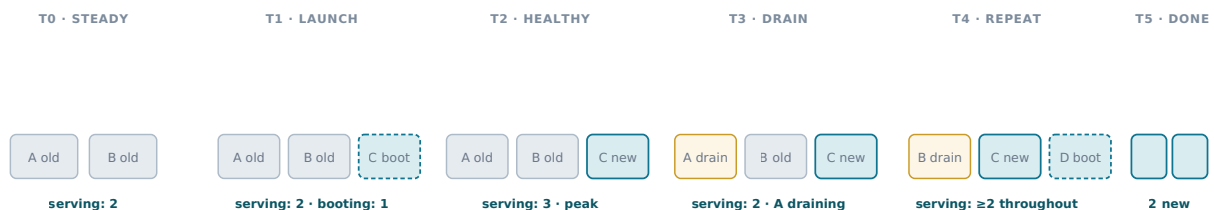
The strategy is a **rolling replacement with launch-before-terminate**, implemented through EC2 Auto Scaling instance refresh. It is not blue/green and not canary. This section explains precisely what it does, and why the alternatives were not chosen.

7.1 What 100 / 100 actually means

Both `asg.tf` and the deploy workflow set `MinHealthyPercentage = 100` and `MaxHealthyPercentage = 100`. Per AWS's documentation, setting the minimum healthy percentage to 100 changes the replacement method to *launch before terminating*: a replacement instance is launched and must pass health checks before the instance it replaces is terminated. Setting both values to 100 additionally constrains the group to replacing **one instance at a time**.

The practical consequence is that serving capacity never drops below the desired count during a deploy. Any other combination permits a dip — with `min = 50` on a two-instance group, capacity can legitimately fall to one instance mid-deploy.

Two-instance group, one deploy — capacity over time



Capacity never falls below the desired count. The transient peak of 3 is what "launch before terminating" costs — a few minutes of one extra instance.

Health gating at T2 uses the target group's /health check, with a 240 s grace period before the ASG will judge a new instance at all.

Draining at T3 respects the target group's deregistration delay — currently the AWS default of 300 s, giving in-flight requests and WebSocket sessions time to finish.

Total wall-clock for a two-instance deploy is typically 8–14 minutes; the pipeline allows up to 30 before declaring a timeout.

Figure 5 — Rolling replacement timeline. The sequence repeats once per instance in the group. A four-instance group takes roughly twice as long as a two-instance one; this is the deliberate cost of never dipping below full capacity.

7.2 Health gating during replacement

CONTROL	VALUE	EFFECT
health_check_type	ELB	Health is the target group's verdict, not merely "is the EC2 instance running". An instance whose container has crashed fails this; it would pass an EC2 -type check.
health_check_grace_period	240 s	How long a new instance is exempt from health judgement. Raised from 60 s after the observability sidecars pushed real time-to-healthy past the old limit and healthy instances were being cycled.
Target group interval	15 s	Probe frequency.
healthy_threshold	3	Three consecutive passes before traffic is sent — a minimum of 45 s after the grace period.
unhealthy_threshold	3	Three consecutive failures before removal, tolerating transient blips.
Deregistration delay	300 s (AWS default — not set explicitly)	Drain window for in-flight work before a target is fully removed.

7.3 Why not blue/green

Blue/green stands up a complete parallel environment, shifts traffic at the load balancer, and keeps the old environment ready for an instant switch back. It is the right answer in several situations. None of them currently apply here, and one of them actively argues against it:

DIMENSION	ROLLING REFRESH (CURRENT)	BLUE/GREEN
Extra compute during deploy	One additional instance, for a few minutes	A full duplicate fleet, for the whole cutover window
Rollback speed	Minutes — requires a second rolling refresh	Seconds — flip the listener back
Database compatibility	Both versions run simultaneously; migrations must be backward-compatible	Same constraint. Both fleets share one database — blue/green does not solve this
Long-lived WebSockets	Drains per instance, gradually, over the deregistration delay	Cutover disconnects every socket on the blue fleet at once
Moving parts	One AWS-native primitive already in use for scaling	Duplicate target groups, listener-swap automation, environment-tracking state

The database row is the decisive one. The most commonly cited benefit of blue/green — instant, safe rollback — is materially weaker here because both fleets would share a single RDS instance. A schema change that breaks the old version breaks it whether that version is running on the green fleet or the blue one. The forward-only migration constraint documented in §6.1 is a property of sharing one database, not a property of rolling deploys, and switching strategies would not remove it.

The WebSocket row is the second. Ocar's traffic is dominated by long-lived Socket.io connections carrying live driver locations. A rolling replacement disconnects a fraction of clients per instance, spread over the deploy, and each reconnects to a healthy target within seconds. An atomic blue/green cutover disconnects every connected rider and driver simultaneously.

WHEN TO REVISIT

Blue/green becomes worth the additional machinery when deploys are frequent enough that multi-minute rollback is a genuine business cost, or when the fleet is large enough that a rolling replacement takes more than about half an hour. Neither is true at two to four instances. Canary — routing a weighted percentage to a new version via ALB weighted target groups — is the more natural next step, and would layer onto the existing setup without replacing it.

7.4 Deploys triggered by infrastructure changes

There is a second, easily-missed path to a rolling replacement. The ASG's `launch_template` block references `aws_launch_template.api.latest_version` — a real numeric version, not the literal string `$Latest`. Any Terraform change that produces a new launch template version (a new AMI, a different instance type, an edited boot script) therefore appears as a genuine diff on the ASG, and the `instance_refresh` block causes `terraform apply` to roll the whole fleet automatically.

IMPORTANT

This means **`terraform apply` can restart your entire production fleet**. It is intended behaviour — without it, a launch template change would silently apply only to instances that happened to be replaced later for unrelated reasons. But it makes reading the plan output before applying non-optional. If the plan shows a change to `aws_launch_template.api`, expect a full rolling replacement.

Request distribution and routing rules

One ALB, one listener, one target group, one rule. The routing configuration is deliberately minimal — path-based routing and per-service target groups would only be justified if the fleet served more than one application, which it does not.

8.1 Listeners

PORT	PROTOCOL	ACTION
80	HTTP	Unconditional <code>HTTP_301</code> redirect to port 443. No plaintext traffic ever reaches a target.
443	HTTPS	TLS terminates here using the ACM certificate. Default action forwards to the API target group. Policy: <code>ELBSecurityPolicy-TLS13-1-2-Res-PQ-2025-09</code> — TLS 1.3 with post-quantum key exchange, restricted cipher set.

`drop_invalid_header_fields` is enabled on the load balancer. This mitigates HTTP request-smuggling and desync attacks by discarding malformed headers before they reach a target. It is off by default in AWS and is worth confirming stays on.

8.2 Target group and algorithm

Target type	<code>instance</code> — registered by the ASG automatically, never by hand
Port / protocol	<code>4000</code> / HTTP (TLS already terminated at the ALB)
Routing algorithm	Round-robin — the ALB default; not overridden
Stickiness	Not enabled — deliberate, see below
Health check	<code>GET /health</code> , expects HTTP 200, every 15 s, 5 s timeout, 3 up / 3 down
Deregistration delay	300 s (AWS default, not set explicitly in Terraform)

Why there is no session stickiness

This is the single most counter-intuitive routing decision in the system, and it is correct. Socket.io is configured with `transports: ['websocket']` only — HTTP long-polling is disabled entirely (`api/src/websocket/socket.server.ts`).

Long-polling is what normally forces sticky sessions: each poll is a separate HTTP request, and every one must land on the instance holding that client's Engine.IO session. A pure WebSocket upgrade has no such requirement — the connection is pinned to one instance

for its entire lifetime by the TCP connection itself. Enabling stickiness would add a cookie and reduce distribution quality for zero benefit.

Cross-instance delivery is handled at the application layer instead. The `@socket.io/redis-adapter` publishes every room emit through Valkey, so `io.to('ride:123').emit(...)` reaches that ride's participants regardless of which instance each is connected to.

ALB idle timeout and Socket.io heartbeats

The ALB's idle timeout is at its default of 60 seconds — a connection with no bytes in either direction for 60 seconds is closed. Socket.io's server-side `pingInterval` is 25 seconds, so an idle WebSocket still exchanges Engine.IO heartbeat frames roughly every 25 seconds, comfortably inside the window. **These two values are coupled.** Raising `pingInterval` above 60 seconds, or lowering the ALB idle timeout below roughly 30, would start dropping idle connections. Change either only with the other in mind.

8.3 Maintenance mode

`alb.tf` defines a `maintenance_mode` boolean. Setting it creates a priority-1 listener rule matching `/*` that returns a fixed JSON 503 response, bypassing the target group entirely:

```
terraform apply -var="environment=prod" -var maintenance_mode=true
```

This is the mechanism to use when the API fleet itself may be unavailable — during a botched instance refresh, for example — and the application-level Redis-backed maintenance flag cannot run because nothing is running. Because ELB health checks probe registered targets directly rather than routing through listener rules, this rule does not interfere with health checking, and instances continue to be evaluated normally while it is active.

8.4 Full request path

#	HOP	WHAT HAPPENS
1	Client	Resolves <code>ocar-api.clienttesting.in</code> .
2	Cloudflare	DNS. Proxy/WAF features are available but the ACM validation record must remain DNS-only, not proxied.
3	ALB :443	TLS termination; malformed headers dropped; forwarded to the target group.
4	Target selection	Round-robin across healthy targets in both AZs.
5	EC2 :4000	Security group permits this only from the ALB's security group.
6	<code>ocar_api</code>	Express middleware chain: Helmet, CORS, rate limiting (Redis-backed, so limits are fleet-wide), body parsing, routing.
7	Data layer	RDS over TLS (<code>rds.force_ssl = 1</code>); Valkey over <code>rediss://</code> .

Rate limiting deserves a note: it uses `rate-limit-redis` rather than in-memory counters. With in-memory state, a limit of 100 requests per minute across four instances is effectively 400. Backing it with Valkey makes the limit hold across the fleet, and is a prerequisite for horizontal scaling that is easy to overlook.

SECTION 09

Capacity: scaling out and in

Capacity moves three ways: automatically, within the configured bounds; manually, for a planned event; and structurally, by changing the bounds themselves. Each has a different mechanism, a different blast radius, and a different reversal path.

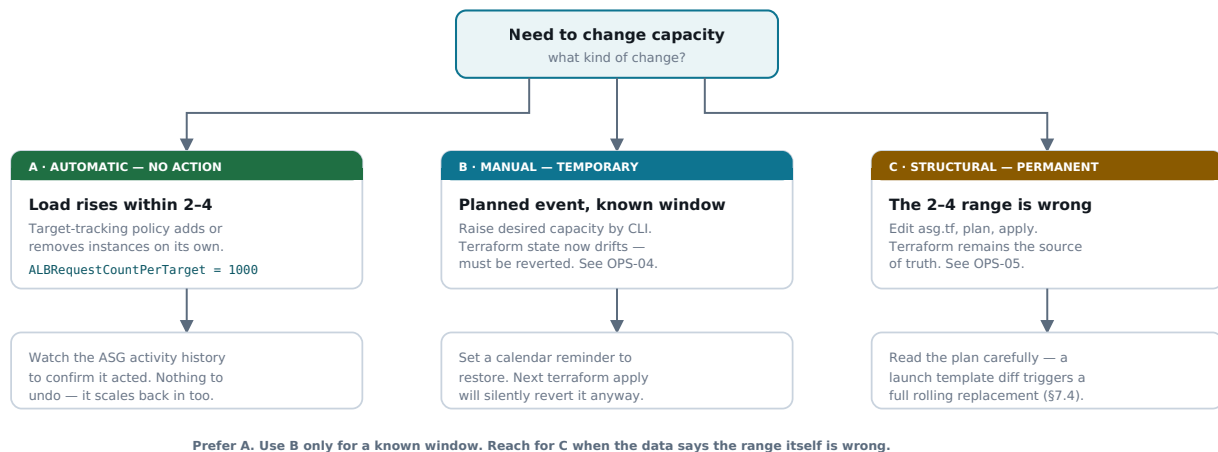


Figure 6 — Choosing a scaling path. The distinction that matters operationally is whether Terraform still describes reality afterwards. Path B deliberately creates drift and must be reversed; path C does not.

9.1 The automatic policy

`aws_autoscaling_policy.request_count_tracking` is a target-tracking policy on the predefined metric `ALBRequestCountPerTarget` with a target value of `1000`. AWS manages the underlying CloudWatch alarms; there are no step adjustments to configure.

READ THIS CAREFULLY — THE UNIT IS NOT PER SECOND

`ALBRequestCountPerTarget` is a *count per target over the metric period*, and target tracking evaluates it on a **one-minute** period. A target value of 1000 therefore means **1000 requests per target per minute — roughly 16.7 requests per second**, not 1000 requests per second. Misreading this by a factor of sixty in either direction produces either a fleet that never scales or one that scales constantly.

The value of 1000 is an estimate, explicitly labelled as such in the code comment. It has not been validated against real traffic. Retuning it is one of the primary deliverables of the load test in §16.

Why request count rather than CPU

For a request-driven Node.js API, CPU is a lagging indicator — by the time CPU is saturated, the event loop is already queuing and latency has already degraded. Request count per target rises before that, giving the roughly four to five minutes a new instance needs to boot, pull images and pass health checks. On `t3` burstable instances there is a second reason: CPU credit exhaustion causes throttling that looks like low CPU utilisation, so a CPU-based policy can fail to scale precisely when the fleet is most degraded.

DISCREPANCY ON RECORD

The client-facing scale-readiness report dated 4 August 2026 states an autoscaling policy of *min 3, max 5, scale on CPU > 70% sustained 3 min*. The deployed configuration is *min 2, max 4, on request count*. The deployed version reflects later analysis and is the better choice for the reasons above, but the two documents disagree and the report has not been revised. Recorded as G-11.

9.2 Scaling boundaries and their consequences

SETTING	VALUE	CONSEQUENCE
<code>min_size</code>	2	Never fewer than two instances, across two availability zones. This is the redundancy floor — one instance may fail entirely without an outage.
<code>desired_capacity</code>	2	Steady-state count. The scaling policy moves this within the min/max bounds.
<code>max_size</code>	4	Hard ceiling. Traffic beyond what four instances absorb degrades rather than scaling — this is the number to raise first under real surge.

TERRAFORM AND DESIRED_CAPACITY

`desired_capacity` is managed by Terraform *and* mutated at runtime by the scaling policy. There is no `lifecycle { ignore_changes }` on it. In practice this means that if the policy has scaled the group to 3 and you then run `terraform apply`, Terraform will see drift and scale it back to 2. This is usually harmless — the policy will immediately scale back out if load warrants — but it can cause a brief capacity dip during an apply performed under load. Avoid routine applies during peak traffic.

9.3 Where the ceiling actually is

Adding instances stops helping once a shared dependency saturates. In order of expected constraint, based on the architecture rather than measurement:

RESOURCE	CURRENT CAPACITY	SYMPTOM WHEN IT BINDS
Valkey pub/sub	One <code>cache.t4g.micro</code> node	Real-time location updates lag or drop; Socket.io fan-out latency climbs. Expected to bind before app CPU.
RDS connections	<code>db.t4g.small</code> , roughly 225 <code>max_connections</code>	Connection acquisition timeouts under load. Each instance holds its own pool, so the total scales with fleet size.
RDS CPU / IOPS	2 vCPU burstable, gp3	Query latency rises across the board; <code>log_min_duration_statement</code> starts logging heavily.
EC2 CPU credits	<code>t3.medium</code> burstable	Sustained load exhausts credits and throttles to baseline — a silent performance cliff, not a clean failure. Monitor <code>CPUCreditBalance</code> .
ASG ceiling	4 instances	Latency rises with no scaling activity in the ASG history.

Note the ordering: the app tier is not expected to be the first constraint. Raising `max_size` in response to a Valkey or RDS bottleneck adds cost without adding capacity, and will make the database constraint worse by adding connection pools. Confirm which resource is actually saturated before scaling the fleet.

SECTION 10

Changing instance type and size

Instance type is a single Terraform variable. The mechanics are trivial; the judgement about *which* type, and the awareness that applying it restarts the fleet, are the parts that matter.

10.1 Where it is defined

`var.instance_type` in `infra/terraform/variables.tf`, default `t3.medium`. It is consumed in exactly one place — `aws_launch_template.api` in `launch-template.tf`. Nothing else references it, so changing it has one direct effect: a new launch template version.

Because the ASG pins `version = aws_launch_template.api.latest_version`, that new version is a real Terraform diff on the ASG, and the `instance_refresh` block turns the apply into a rolling replacement of the whole fleet (§7.4). This is the intended behaviour — it is how the change actually reaches running instances — but it must be expected rather than discovered.

10.2 Understanding the current choice

`t3.medium` is 2 vCPU / 4 GiB from AWS's burstable family. Burstable instances earn CPU credits while running below a baseline utilisation and spend them to burst above it. For `t3.medium` the baseline is 20% of each vCPU.

This suits the workload's shape: thousands of mostly-idle WebSocket connections punctuated by short bursts of work when GPS pings arrive or a ride is booked. Paying for fixed 24/7 performance to cover load that only spikes would be waste.

THE RISK THIS CREATES

Sustained high traffic — precisely a marketing surge — can exhaust the credit balance. The instance then throttles to baseline. The failure mode is a silent performance cliff, not a clean scale-out signal, and it can defeat CPU-based autoscaling (which is one reason this fleet scales on request count instead). Two mitigations exist: enable **Unlimited** burst mode, so the instance bursts past its credit balance for a small per-vCPU-hour surcharge instead of throttling; or move to a fixed-performance family. Unlimited mode is **not currently enabled** — recorded as G-16.

10.3 Candidate types

TYPE	VCPU / RAM	FAMILY	WHEN IT IS THE RIGHT ANSWER
<code>t3.medium</code> current	2 / 4 GiB	Burstable x86	Present workload: low baseline, spiky. Cheapest option that meets the shape.
<code>t3.large</code>	2 / 8 GiB	Burstable x86	Memory pressure without CPU pressure — the container limit is already 2 GiB, so headroom is thin if a leak appears.
<code>m5.large</code> / <code>m6i.large</code>	2 / 8 GiB	Fixed x86	The load test shows credits draining. No credit mechanic, predictable performance, roughly double the cost.
<code>c6i.large</code>	2 / 4 GiB	Compute- optimised	CPU-bound rather than memory-bound. Unlikely for this workload.
<code>t4g.medium</code> / <code>m6g.large</code>	2 / 4–8 GiB	Graviton (ARM)	Roughly 20% cheaper for equivalent performance — but requires an ARM image and an ARM AMI . See the warning below.

ARCHITECTURE CHANGE — NOT A DROP-IN

Switching to any Graviton type (`t4g` , `m6g` , `c7g`) is **not** a one-variable change. Three things must move together, or instances will boot and fail to run the container:

- The AMI filter in `launch-template.tf` is hardcoded to `ubuntu-noble-24.04-amd64-server-*` and must become `arm64` .
- The container image is built for a single architecture. `build-push` in `deploy.yml` must produce a multi-architecture manifest (`platforms: linux/amd64,linux/arm64` in `docker/build-push-action`), or the pull succeeds and `exec` fails.
- Every sidecar image — Alloy, node-exporter, cAdvisor, postgres-exporter — must have an `arm64` variant at the pinned tag. Verify each before switching.

Do this as its own deliberate change, validated on staging first. It is worthwhile — the saving is real — but it is a project, not an edit.

10.4 Sizing the container limits alongside the instance

Changing the instance type does not change the container's limits. `ocar_api` is capped at `mem_limit: 2g` , `cpus: 1.5` in `docker-compose.prod.yml` regardless of what it runs on. Moving to an 8 GiB instance without raising `mem_limit` leaves 6 GiB unusable by the application.

Node 22 is container-aware and sizes its V8 heap from the cgroup limit, so no `--max-old-space-size` flag is needed — but it reads the `container` limit, not the instance's memory. The two must be changed together. Note also that `docker-compose.prod.yml` reaches instances via an SSM parameter sourced from the repo file, so a change to it requires a `terraform apply` to update the parameter and a fleet roll to take effect — it does not ride along with an application deploy.

SIZING FROM DATA, NOT GUESSWORK

The current limits are explicitly a safety ceiling, not a tuned value. Their purpose is to stop an unbounded leak from taking down the whole instance by letting the kernel OOM-killer pick an arbitrary victim container. The cAdvisor "Per-Container Resource Usage" row on the `ocar-overview` Grafana dashboard reports working set, percentage of limit, CPU cores and OOM events per container. Size the final values from observed p95/p99 there once real or load-test traffic exists.

Configuration and secret management

All runtime configuration lives in SSM Parameter Store. Instances read it once, at boot. Nothing is baked into the image, and no secret is stored in Terraform state or in the repository.

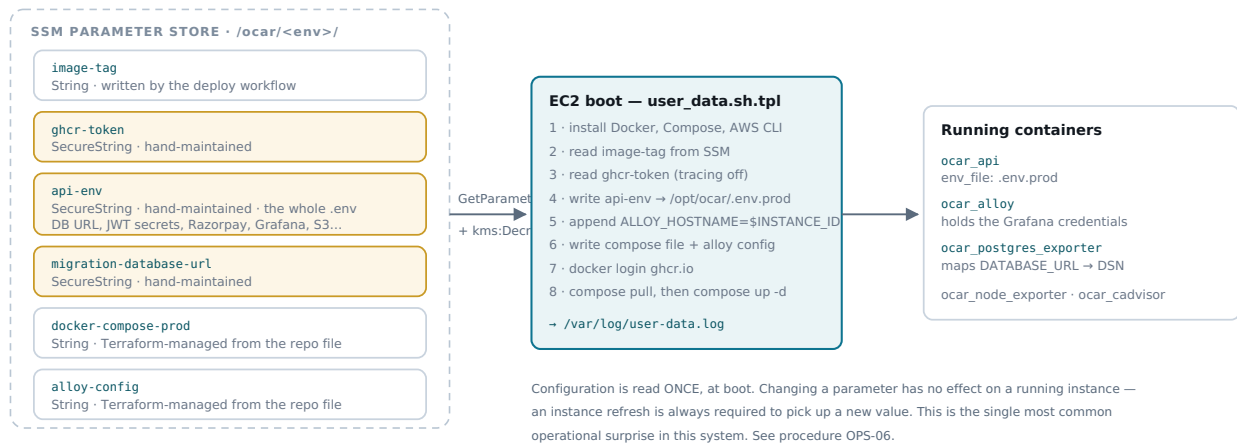


Figure 7 — Configuration flow. Amber marks `SecureString` parameters, encrypted with the `alias/aws/ssm` KMS key. Two parameters are Terraform-managed and sourced directly from repository files; four are hand-maintained.

11.1 Parameter inventory

PARAMETER	TYPE	MANAGED BY	CONTENTS
image-tag	String	Deploy workflow	Which image version instances boot. The rollback anchor.
ghcr-token	SecureString	Manual	GHCR pull credential.
api-env	SecureString	Manual	The application's entire environment file.
migration-database-url	SecureString	Manual	Direct database URL for the migration step.
docker-compose-prod	String, Advanced tier	Terraform	Sourced from the repo file at apply time.
alloy-config	String, Advanced tier	Terraform	Sourced from the repo file at apply time.

Both Terraform-managed parameters use the Advanced tier because they exceed the Standard tier's 4096-character limit — the Compose file crossed it when cAdvisor and the resource limits were added, and the Alloy config when the postgres-exporter scrape block was added. Advanced tier costs \$0.05 per parameter per month.

11.2 How secrets stay out of logs

The boot script runs under `set -x`, which traces variable *assignments*, not just commands. A plain `GHCR_TOKEN=$(aws ssm get-parameter ...)` would print the resolved secret into `/var/log/user-data.log`. The script therefore disables tracing around both the assignment and the subsequent `docker login`, and re-enables it immediately after. This is worth preserving if the script is ever edited.

11.3 The known ergonomic problem

WHOLE-VALUE OVERWRITE

SSM supports only whole-value replacement. Changing one line in `api-env` means pulling the entire decrypted blob to a local file, editing it by hand, and pushing the whole thing back. During the Redis-to-Valkey migration on 12 August 2026 this caused a real incident: a hand edit changed the `REDIS_URL` scheme but missed the hostname, because the whole value has to be reviewed at once instead of editing one key.

A per-key secrets manager would remove this failure mode. It is on the backlog and deliberately not urgent — SSM `SecureString` is a legitimate credential store, the problem is ergonomics rather than security. Procedure `OPS-06` gives the safe sequence for editing it today.

SECTION 12

Data layer

12.1 RDS PostgreSQL

Engine	PostgreSQL 18.4 with PostGIS
Class / storage	<code>db.t4g.small</code> · gp3, 20 GB auto-scaling to 100 GB, encrypted at rest
Availability	Single-AZ. No read replica. accepted at current scale
Backups	7-day automated retention; final snapshot on destroy in prod only

Deletion protection	Enabled in prod; disabled in staging
Network	<code>publicly_accessible = false</code> ; reachable only from the EC2 security group
Master password	Generated and rotated by AWS in Secrets Manager — never in Terraform state
Observability	Performance Insights (7-day retention); <code>postgresql</code> and <code>upgrade</code> logs to CloudWatch

`manage_master_user_password = true` means the password never appears in state or in a `.tfvars` file. Retrieve it once after apply to construct the connection string that goes into `api-env`:

```
aws secretsmanager get-secret-value \
  --secret-id ${terraform output -raw rds_master_user_secret_arn} \
  --query SecretString --output text
```

Parameter group — three settings that cannot be fixed later without downtime

PARAMETER	VALUE	APPLY METHOD	WHY
<code>rds.logical_replication</code>	<code>1</code>	pending-reboot	Required for RDS to act as a logical replication subscriber during the Neon-to-RDS migration.
<code>shared_preload_libraries</code>	<code>pg_stat_statements</code>	pending-reboot	RDS does not preload this by default. Without it, <code>CREATE EXTENSION</code> fails outright and the postgres-exporter scrape breaks permanently.
<code>rds.force_ssl</code>	<code>1</code>	dynamic	RDS does not enforce TLS out of the box; unencrypted connections would otherwise be silently accepted.
<code>log_min_duration_statement</code>	<code>500</code>	dynamic	Logs queries slower than 500 ms to CloudWatch. Without it the log export has nothing duration-based to record.

The two `pending-reboot` parameters are static and require an instance reboot to take effect. They must be applied and the instance rebooted *before* the operations that depend on them are attempted.

12.2 ElastiCache Valkey

Engine	Valkey 8.0, single node (<code>num_cache_clusters = 1</code> , failover disabled)
Node type	<code>cache.t4g.micro</code>
Eviction policy	<code>noeviction</code> — required , see below
Encryption	At rest and in transit; transit mode currently <code>preferred</code>
Roles	Socket.io pub/sub adapter · BullMQ queues · rate-limit counters · OTP and session cache

WHY NOEVICTION IS NOT OPTIONAL

BullMQ stores job payloads in Redis. The ElastiCache default parameter group uses `volatile-lru` , which evicts keys under memory pressure — meaning queued jobs (notifications, GPS flushes, settlements) can silently disappear. This exact misconfiguration was observed on the previous Redis Cloud instance. Do not change this parameter.

TWO OPEN ITEMS

Transit encryption is in `preferred` mode, which accepts both `rediss://` and plaintext `redis://` . This was the correct choice for migrating without a hard cutover, but it should be flipped to `required` once every instance is confirmed on `rediss://` . Recorded as

G-12 .

The cache is a single point of failure. There is no replica and automatic failover is disabled. If the node fails, Socket.io fan-out, BullMQ and rate limiting all stop. Cache contents are genuinely disposable, so the recovery is a node replacement rather than a data loss event — but it is an outage. Adding a replica is a one-line change when the availability requirement justifies the cost.

SECTION 13

Observability

One agent per instance ships logs, metrics and traces to one platform. Grafana Alloy (`ocar_alloy`) reads from the Docker socket and from three exporters, and forwards to Grafana Cloud — Loki for logs, Mimir for metrics, Tempo for traces.

13.1 Signals

SIGNAL	SOURCE	DESTINATION	NOTES
Logs	Pino JSON on container stdout	Loki	Exactly four indexed labels — <code>container</code> , <code>service</code> , <code>env</code> , <code>host</code> . Everything else, including <code>requestId</code> , stays in the JSON body and is queried rather than indexed.
App metrics	<code>prom-client</code> at <code>/metrics</code>	Mimir	<code>http_request_duration_seconds</code> , <code>pg_pool_connections</code> , <code>bullmq_queue_job_counts</code> .
Host metrics	<code>node-exporter</code>	Mimir	CPU, memory, disk, network per instance.
Container metrics	<code>cAdvisor</code>	Mimir	Aggressively filtered — see below.
Database metrics	<code>postgres-exporter</code>	Mimir	Includes <code>pg_stat_statements</code> . Every instance reports identical numbers; dashboards use <code>max()</code> not <code>sum()</code> .
Traces	OpenTelemetry SDK → Alloy OTLP	Tempo	The app sends to <code>alloy:4318</code> , never directly to Grafana Cloud, so credentials live only in Alloy.

Log and trace correlation works because the tracing SDK's `trace_id` feeds Pino's mixin — every log line carries the ID of the span it was emitted within, so a trace in Tempo can be pivoted to its logs in Loki.

13.2 Cardinality controls — read before editing `config.alloy`

Two incidents shaped the current relabelling rules, and both are easy to reintroduce:

- **18 August 2026.** `cAdvisor`'s roughly 100 metric families, multiplied across a rolling replacement, tipped the Grafana Cloud tenant's active-series cap. The fix keeps only five metric families, grepped from the dashboard that actually queries them.
- **19 August 2026.** The deploy pipeline's migration container was launched with a random name, so `cAdvisor` discovered a new container on every deploy and permanently inflated the series count. The fix is two-part: the container is now explicitly named `db_migrate` , and Alloy keeps only containers matching `ocar_*` .

A `labelkeep` rule then retains only `__name__` , `name` and `host` , which eliminates churn from container IDs and image tags in one rule rather than a drop-list that must be maintained.

PRACTICAL CONSEQUENCE

Adding a container to `docker-compose.prod.yml` whose name does not start with `ocar_` means it will not appear in metrics at all. That is usually what you want, but it is silent — nothing warns you.

13.3 Alerting

Managed as code in `infra/terraform/observability/`, with its own state at `observability/terraform.tfstate`. Applied locally rather than from CI, because it changes rarely enough that a pipeline credential is not justified.

RULE GROUP	COVERS
<code>ocar-static-thresholds</code>	Error rate on <code>/api/v1/rides.*</code> and <code>/api/v1/payments.*</code> , evaluated every 60 s.
<code>ocar-billing-usage</code>	Grafana Cloud free-tier consumption at 85% of cap — active series, log volume, trace volume.
SLOs	Ride request success rate, 98% objective over 30 days. Explicitly a starting value, not yet backed by real traffic history.
Synthetic check	<code>GET /health</code> from Mumbai and Singapore probes every 60 s, with a 14-day TLS-expiry threshold.

DO NOT HAND-EDIT IN THE GRAFANA UI

`grafana_notification_policy` is a whole-tree resource — applying it replaces the entire policy. Any change made in the Grafana interface will be silently destroyed by the next `terraform apply`. All notification routing changes must go through the Terraform module.

SECTION 14

Operational procedures

Twelve numbered procedures covering every routine and emergency operation. Each states its blast radius and how to reverse it. Shared prerequisites: AWS CLI authenticated against `ap-south-1`, Terraform 1.15.8, repository checked out, working directory `infra/terraform` for anything Terraform-related.

Resolving the placeholders

```
# ASG name and ALB DNS name – also stored as GitHub Actions repository variables
terraform output -raw autoscaling_group_name
terraform output -raw alb_dns_name

# or without Terraform:
aws autoscaling describe-auto-scaling-groups \
  --query 'AutoScalingGroups[?contains(AutoScalingGroupName,`ocar`)].AutoScalingGroupName' --output text
```

Blast radius: production fleet rolled one instance at a time. No capacity dip. **Reversal:** automatic on failure; manual via OPS-02.

- 1 Merge to `main`. Nothing else is required — the pipeline is fully automatic.
- 2 Watch the **CI** workflow. All seven jobs must pass or Deploy never starts.
- 3 Watch the **Deploy** workflow. If the *Detect relevant changes* job reports `relevant=false`, the change did not touch anything that reaches the image and no deploy is needed — this is correct behaviour, not a fault.
- 4 Observe the rollout while it runs:

```
watch -n 15 'aws autoscaling describe-instance-refreshes \
  --auto-scaling-group-name <ASG_NAME> --max-records 1 \
  --query "InstanceRefreshes[0].
  {Status:Status,Pct:PercentageComplete}" --output table'
```

- 5 Confirm afterwards:

```
curl -s https://ocar-api.clienttesting.in/health | jq
curl -s https://ocar-api.clienttesting.in/api/v1/geo/cities |
jq '.[0]'
```

If a migration is involved: it runs before any instance is replaced and gates the rollout. It must be backward-compatible with the currently-running code, because the old version keeps serving traffic throughout — see §6.1.

Blast radius: full fleet roll. **Reversal:** repeat with the newer tag.

- 1 Identify the target tag. Tags are `sha-<short-sha>` and are listed on the GHCR package page, or in the *Docker metadata* step of a previous Deploy run.

- 2 Record what is currently deployed, so this step is itself reversible:

```
aws ssm get-parameter --name /ocar/prod/image-tag --query 'Parameter.Value' --output text
```

- 3 Point the parameter at the target version:

```
aws ssm put-parameter --name /ocar/prod/image-tag \
  --type String --value "sha-abc1234" --overwrite
```

- 4 Roll the fleet so instances actually pick it up:

```
aws autoscaling start-instance-refresh \
  --auto-scaling-group-name <ASG_NAME> \
  --preferences '{"MinHealthyPercentage": 100, "MaxHealthyPercentage": 100}'
```

- 5 Verify as in OPS-01 step 5.

MIGRATIONS ARE NOT REVERTED

Schema changes are forward-only. Rolling the image back does not undo a migration. If the migration is the fault, the old code will now be running against a newer schema — which may be worse than the problem you are fixing. Fix forward with a new migration instead.

Blast radius: every request receives HTTP 503 immediately. **Reversal:** re-apply with the flag false.

1 Enable:

```
terraform apply -var="environment=prod" -var  
maintenance_mode=true
```

2 Confirm the fixed response is being served:

```
curl -i https://ocar-api.clienttesting.in/api/v1/geo/cities
```

Expect 503 with "code": "MAINTENANCE_MODE".

3 Disable when finished:

```
terraform apply -var="environment=prod" -var  
maintenance_mode=false
```

Health checks are unaffected — ELB probes registered targets directly rather than through listener rules, so instances continue to be evaluated normally while the rule is active. Use this when the fleet itself may be unavailable and the application-level maintenance flag cannot run.

Blast radius: adds instances and cost. **Reversal:** mandatory — see step 4.

- 1 Check the current bounds:

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names <ASG_NAME> \
  --query 'AutoScalingGroups[0].{Min:MinSize,Desired:DesiredCapacity,Max:MaxSize}' --output table
```

- 2 Raise desired capacity. It must stay within the existing min/max, so raise `--max-size` in the same call if you need to exceed 4:

```
aws autoscaling update-auto-scaling-group \
  --auto-scaling-group-name <ASG_NAME> \
  --min-size 4 --desired-capacity 4 --max-size 6
```

Raising `--min-size` too is deliberate: it stops the target-tracking policy from immediately scaling back in while traffic is still low, just before the event starts.

- 3 Confirm the new instances reach `InService` — typically four to five minutes:

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names <ASG_NAME> \
  --query 'AutoScalingGroups[0].Instances[{Id:InstanceId,State:LifecycleState,Health:HealthStatus}]' \
  --output table
```

- 4 **Restore afterwards.** Either repeat step 2 with the original values, or run `terraform apply`, which will reset them to whatever `asg.tf` declares.

THIS CREATES TERRAFORM DRIFT ON PURPOSE

Terraform still believes the group is 2/2/4. Anyone running `terraform apply` for an unrelated reason will silently revert your change — possibly mid-event. Announce it, and set a reminder to restore it yourself.

Blast radius: capacity change only — no instance replacement, provided the launch template is untouched. **Reversal:** revert the commit and re-apply.

- 1 Edit `min_size`, `desired_capacity` or `max_size` in `infra/terraform/asg.tf`.
- 2 Open a pull request. `terraform-plan.yml` posts the plan as a PR comment automatically for any change under `infra/**`.
- 3 Read the plan. It should show **only** the ASG changing. If it also shows `aws_launch_template.api` changing, expect a full rolling replacement (\$7.4).
- 4 Merge, then apply from a workstation — infrastructure does not auto-apply on merge:

```
pnpm infra:prod:init    # once per session, or after
                        switching environments
pnpm infra:prod:plan
pnpm infra:prod:apply
```

- 5 Consider whether `target_value` on the scaling policy also needs adjusting — raising `max_size` without retuning the threshold changes the ceiling but not the trigger point.

Blast radius: full fleet roll at step 5. **Reversal:** keep the pre-edit copy until verified.

- 1 Pull the current value to a file. Do this in a directory that is not the repository:

```
cd $(mktemp -d)
aws ssm get-parameter --name /ocar/prod/api-env --with-decryption \
  --query 'Parameter.Value' --output text > api-env.txt
cp api-env.txt api-env.backup.txt # rollback copy
```

- 2 Edit `api-env.txt`. Change only the line you intend to change — the entire blob is replaced, so an accidental edit elsewhere ships silently.
- 3 Diff before pushing. This step is not optional; it is what would have caught the 12 August 2026 `REDIS_URL` incident:

```
diff api-env.backup.txt api-env.txt
```

- 4 Push the whole value back:

```
aws ssm put-parameter --name /ocar/prod/api-env --type SecureString \
  --value "$(cat api-env.txt)" --overwrite
```

- 5 Roll the fleet — running instances read SSM only at boot and will not otherwise notice:

```
aws autoscaling start-instance-refresh \
  --auto-scaling-group-name <ASG_NAME> \
  --preferences '{"MinHealthyPercentage": 100, "MaxHealthyPercentage": 100}'
```

- 6 Verify, then destroy the plaintext copies:

```
curl -s https://ocar-api.clienttesting.in/health | jq
shred -u api-env.txt api-env.backup.txt 2>/dev/null || rm -f
api-env.txt api-env.backup.txt
```

Blast radius: full rolling replacement. **Reversal:** revert the variable and apply again — another full roll.

- 1 Confirm the target type is the same CPU architecture. If moving to Graviton, stop and read §10.3 — that is a multi-part change, not this procedure.
- 2 Change the default of `instance_type` in `infra/terraform/variables.tf`, or pass it at apply time for a one-off test.
- 3 Plan and read the output. Expect `aws_launch_template.api` to show a new version and the ASG to show a corresponding change:

```
pnpm infra:prod:init
pnpm infra:prod:plan
```

- 4 Consider raising `mem_limit` and `cpus` in `docker-compose.prod.yml` in the same change if you are moving to a larger instance — otherwise the extra memory is unusable (§10.4).
- 5 Apply. This triggers a rolling replacement automatically; there is no separate refresh step:

```
pnpm infra:prod:apply
```

- 6 Watch the refresh as in OPS-01 step 4, then confirm the new type is live:

```
aws ec2 describe-instances \
  --filters "Name=tag:aws:autoscaling:groupName,Values=<ASG_NAME>" \
    "Name=instance-state-name,Values=running" \
  --query 'Reservations[].Instances[].[InstanceId,InstanceType]' --output table
```

- 7 Watch CPU, memory and — if still on a `t3` — `CPUCreditBalance` for at least one full traffic cycle before considering the change settled.

Blast radius: depends entirely on the plan. Read it. **Reversal:** revert and re-apply.

- 1 Make the change in `infra/terraform/*.tf` on a branch.
- 2 Open a pull request. The plan is posted as a PR comment automatically.
- 3 Read the plan against this checklist before merging:
 - Does it show `aws_launch_template.api` changing? → the fleet will roll.
 - Does it show `aws_elasticache_replication_group` being *replaced*? → the cache will be destroyed and recreated. Contents are disposable, but expect an interruption to real-time delivery and queued jobs.
 - Does it show `aws_db_instance` being *replaced*? → **stop**. That is data loss. Deletion protection should prevent it in prod, but do not rely on that.
 - Does it show anything being destroyed that you did not intend to destroy?
- 4 Merge, then apply from a workstation:

```
pnpm infra:prod:init
pnpm infra:prod:plan
pnpm infra:prod:apply
```

Infrastructure never auto-applies on merge. Only the plan runs in CI. This is deliberate — AWS changes are not cheaply reversible the way an application deploy is.

Blast radius: creates a second full environment and its cost. **Reversal:** OPS-10 .

- 1 Create the DNS record for `staging.ocar-api.clienttesting.in` in Cloudflare. ACM validation will block the apply until the validation CNAME resolves publicly — it must be **DNS-only, not proxied**.
- 2 Seed the four hand-maintained SSM parameters under `/ocar/staging/` : `image-tag` , `ghcr-token` , `api-env` , `migration-database-url` . Use non-production credentials throughout — Razorpay test keys, a distinct JWT secret.
- 3 Run the workflow: **Actions** → **Staging Infra (apply/destroy)** → **Run workflow** → **action: apply** . Or locally:

```
pnpm infra:staging:init
pnpm infra:staging:plan
pnpm infra:staging:apply
```
- 4 Point the Cloudflare record at the staging ALB DNS name from the apply output.
- 5 Full step-by-step for the manual, non-Terraform portions: `docs/superpowers/specs/2026-08-14-staging-runbook.md` .

KNOWN BLOCKER

The `ocar-prod-gha-staging` IAM role grants no `rds:*` permissions, but `rds.tf` is part of the shared configuration. A staging apply through the workflow will therefore **fail** when it reaches the database resources. This must be fixed before OPS-09 can complete via CI — see [G-02](#) . A local apply with broader credentials is the current workaround.

Blast radius: destroys the entire staging environment. **Reversal:** OPS-09 from scratch.

- 1 **Verify which environment you are targeting.** Terraform reuses whatever backend the local `.terraform/` directory was last initialised against:

```
terraform init -reconfigure -backend-  
config=staging.backend.hcl  
terraform state list | head
```

- 2 Destroy:

```
pnpm infra:staging:destroy
```

Or the workflow with `action: destroy`.

- 3 Remove the Cloudflare DNS records for the staging hostname.
- 4 Confirm nothing survived — orphaned ElastiCache nodes and RDS instances are the expensive ones:

```
aws rds describe-db-instances --query 'DBInstances[].DBInstanceIdentifier'  
aws elasticache describe-replication-groups --query 'ReplicationGroups[].ReplicationGroupId'
```

NO CONFIRMATION GATE EXISTS

The workflow's `environment: staging` declaration is inert — the GitHub Environment was never created. A `destroy` dispatch runs `terraform destroy -auto-approve` against real infrastructure with no approval step and no undo. See [G-05](#). Always destroy after a load test; staging is not meant to run continuously and costs the same per-instance rate as production.

Blast radius: one instance. **Reversal:** none needed — the ASG relaunches automatically.

- 1 Identify it:

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-  
group-names <ASG_NAME> \  
--query 'AutoScalingGroups[0].Instances[0].  
{Id:InstanceId,State:LifecycleState,Health:HealthStatus,AZ:Av  
ailabilityZone}' \  
--output table
```

- 2 Look before you kill it — the cause is usually in the boot log or the container log:

```
aws ssm start-session --target <INSTANCE_ID>  
sudo tail -100 /var/log/user-data.log  
sudo docker ps -a  
sudo docker logs --tail 200 ocar_api
```

- 3 Terminate it and let the ASG replace it. `--should-decrement-desired-capacity false` is what makes the replacement happen:

```
aws autoscaling terminate-instance-in-auto-scaling-group \  
--instance-id <INSTANCE_ID> \  
--no-should-decrement-desired-capacity
```

- 4 Confirm the replacement reaches `InService` and `Healthy`. Allow at least the 240-second grace period plus three 15-second health checks.

If replacements repeatedly fail to become healthy, the problem is systemic — a bad image, a broken SSM parameter, or an unreachable dependency. Go to [OPS-12](#) rather than cycling instances.

Blast radius: read-only until you choose an action.

- 1 **Locate the failing step** in the Deploy workflow run. The step name maps directly onto Figure 4.
- 2 **Migration step failed** → the rollout never started and production is untouched, still running the previous version. Read the captured stdout/stderr in the workflow log. Fix the migration and merge again. Do not attempt to force the rollout past it.
- 3 **Refresh timed out or failed** → replacement instances are not becoming healthy:

```
aws autoscaling describe-instance-refreshes \
  --auto-scaling-group-name <ASG_NAME> --max-records 1 \
  --query 'InstanceRefreshes[0].
  {Status:Status,Reason:StatusReason,Pct:PercentageComplete}'
```

Then shell into a failing instance and read `/var/log/user-data.log`. The usual causes are a GHCR pull failure (expired `ghcr-token`), a malformed `api-env`, or the application exiting at startup because it cannot reach RDS or Valkey.

- 4 **Health check failed but the refresh succeeded** → instances are healthy at the target group but the ALB is not serving correctly. Check target health directly:

```
aws elbv2 describe-target-health --target-group-arn <TG_ARN>
```

- 5 **Smoke test failed but health passed** → this is the important case. It means the process is alive but the database path is broken, which `/health` does not currently detect (`G-01`). Check RDS availability, the security group, and `DATABASE_URL` in `api-env`.

- 6 **To stop an in-progress refresh:**

```
aws autoscaling cancel-instance-refresh --auto-scaling-group-name <ASG_NAME>
```

Cancelling leaves the group in a mixed state — some instances on the new version, some on the old. Follow immediately with `OPS-02` to converge on a known-good tag.

SECTION 15

Failure modes and recovery

What breaks, what it looks like, and what to do. Ordered by how likely each is to occur.

FAILURE	OBSERVABLE SIGNAL	AUTOMATIC RESPONSE	ACTION
One instance dies	Target group shows one unhealthy target; no user-visible impact	ASG terminates and replaces it; ALB stops routing to it after 3 failed checks	None required. Investigate the cause via OPS-11 if it recurs.
Bad deploy	Deploy workflow red at the health or smoke step	Automatic rollback to the previous tag, plus a second refresh	Confirm the rollback completed. Diagnose with OPS-12 .
Bad migration	Deploy fails at the migration step	Rollout never starts; production untouched	Fix forward. Never attempt to revert a migration.
CPU credits exhausted	CPUCreditBalance near zero; latency rising with no scaling activity	None — request-count scaling may still fire, CPU-based would not	Enable Unlimited mode, or move to a fixed-performance type (OPS-07).
Valkey node fails	Real-time updates stop; BullMQ stalls; rate limiting fails open or closed	None — single node, failover disabled	Replace the node. Cache contents are disposable; this is an outage, not data loss.
RDS unavailable	Smoke test fails; /health reports <code>db: "error"</code> in its body but still returns HTTP 200	None — single-AZ, no replica. The ALB does not detect this	Check the RDS console. Restore from automated backup if required. See G-01 and G-10 .
Entire AZ fails	Half the fleet unhealthy	ALB routes to the surviving AZ; ASG launches replacements there	Monitor. If RDS was in the failed AZ, the database is down until AWS recovers it — single-AZ is an accepted risk.
Grafana Cloud cap exceeded	Billing-usage alert at 85%; metrics or logs start being dropped	Alert fires	Review cardinality (§13.2) before assuming an upgrade is needed. Both prior incidents were cardinality, not genuine volume.
GHCR token expired	Every new instance fails to become healthy; docker	None — the ASG will cycle instances indefinitely	Rotate /ocar/prod/ghcr-token , then refresh (OPS-06 steps 4–5).

FAILURE	OBSERVABLE SIGNAL	AUTOMATIC RESPONSE	ACTION
	login error in user-data.log		

THE PATTERN WORTH INTERNALISING

Every failure the platform recovers from automatically is one where **an instance** is the failing unit. Every failure requiring manual intervention is one where a **shared singleton** fails — the Valkey node, the RDS instance, an SSM parameter. That is the accurate summary of this architecture's resilience profile: the compute tier is genuinely redundant; the data tier is not.

SECTION 16

Load testing

A k6 suite exists at `load-tests/`, built to exercise the specific numbers committed in the scale-readiness report: 5,000–6,000 concurrent connections, GPS pings every three to five seconds, and booking bursts on top.

16.1 Structure

PATH	PURPOSE
<code>seed/generate-test-tokens.js</code>	Mints JWTs directly rather than driving real OTP flows — thousands of logins would cost real money in SMS and hit Fast2SMS rate limits. Creates synthetic riders; reuses real onboarded drivers and stamps their daily verification records.
<code>seed/generate-bulk-ride-history.js</code>	Populates history so read paths are exercised against realistic table sizes.
<code>k6/lib/socketio.js</code>	Hand-rolled Engine.IO v4 / Socket.io v4 client. k6 runs its own JS runtime, so the real <code>socket.io-client</code> package cannot be used.
<code>k6/smoke.js</code>	Single-VU sanity check across every code path. Run before every session.
<code>k6/main.js</code>	The full test — concurrent booking, driver GPS and idle-rider scenarios.
<code>k6/spike.js</code> , <code>k6/accept-race.js</code>	Sudden-surge behaviour, and concurrent ride-acceptance contention.
<code>verify/reconcile.js</code> , <code>verify/query-regression.js</code>	Post-run data-integrity and query-performance verification.

16.2 Method

1. Stand up staging (`OPS-09`) — note the blocker recorded there.
2. Seed data and tokens.
3. Run `smoke.js` privately first. A live run should never be the script's first contact with the environment.
4. Step up in stages rather than starting at target load: roughly 200 → 1,000 → 3,000 → 6,000 connections.
5. Watch, in order of expected constraint: Valkey ops/sec and connections; RDS connection count and Performance Insights; `CPUCreditBalance` ; the cAdvisor per-container row; ALB 5xx rate and p95 latency.
6. Tear staging down afterwards (`OPS-10`).

16.3 What the test is expected to answer

QUESTION	DECISION IT FEEDS
Do <code>t3.medium</code> credits survive sustained load?	Enable Unlimited mode, or move to <code>m5.large</code> .
What is the real per-instance request ceiling?	Retune <code>target_value</code> from 1000 to a measured number.
Does <code>cache.t4g.micro</code> hold the pub/sub fan-out?	Whether to increase the Valkey node size.
Does <code>db.t4g.small</code> hold the connection count?	Instance class, and whether a pooler is needed.
Is <code>max_size = 4</code> sufficient?	The scaling bounds in <code>asg.tf</code> .

Until this test runs, the capacity figures in the scale-readiness report are engineering estimates reasoned from the stack — not measurements. That distinction is stated in the report itself and is repeated here because it matters for sign-off.

SECTION 17

Known gaps and risk register

Eighteen items, each verified against the current codebase. Severity reflects operational consequence, not effort to fix. Several are deliberate trade-offs with sound reasoning; those are marked as accepted rather than outstanding.

17.1 Summary

REF	SEVERITY	ISSUE	STATUS
G-01	High	<code>/health</code> returns 200 when the database is down	Outstanding
G-02	High	Staging IAM role cannot create RDS — staging apply fails	Outstanding
G-05	High	Unguarded <code>terraform destroy</code> on staging	Blocked on repo admin
G-06	High	No branch protection; no deploy approval gate	Blocked on repo admin
G-13	High	Minimal automated test coverage; no coverage gate	Outstanding
G-14	High	Admin TOTP enforcement disabled by a hardcoded bypass	Outstanding
G-03	Medium	Plan role lacks <code>rds:Describe*</code>	Outstanding
G-09	Medium	Scaling threshold is an unvalidated estimate	Awaiting load test
G-10	Medium	Single-AZ database, single-node cache	Accepted at current scale
G-11	Medium	Scale-readiness report contradicts deployed scaling config	Outstanding — documentation
G-12	Medium	Cache transit encryption is <code>preferred</code> , not <code>required</code>	Outstanding
G-15	Medium	Dependency scanning gates on <code>CRITICAL</code> only	Accepted, revisitable
G-16	Medium	<code>t3</code> Unlimited burst mode not enabled	Awaiting load test
G-04	Low	IAM policy comment is stale and misleading	Outstanding — documentation
G-07	Low	ASG sizing hardcoded rather than variable	Outstanding
G-08	Low	Network resource names not environment-scoped	Outstanding — cosmetic
G-17	Low	Container limits shipped in code but not applied to running instances	Outstanding
G-18	Info	Deregistration delay left at the AWS default	Accepted

17.2 Detail on the high-severity items

G-01 — The health endpoint cannot fail

`api/src/app.ts` checks database and Redis connectivity, reports the result in the response body, and then unconditionally returns HTTP 200:

```
const [dbOk, redisOk] = await Promise.all([testConnection(), testRedis()])
res.status(200).json({ status: 'ok', db: dbOk ? 'ok' : 'error', ... })
```

The ALB target group matches on `200`. An instance whose database connection has failed therefore remains **healthy** and continues receiving traffic indefinitely. The ASG will not replace it. The deploy pipeline's health check will pass.

The smoke test against `/api/v1/geo/cities` is currently the only automated check that would catch this, and it runs once per deploy rather than continuously. Suggested fix: return 503 when `dbOk` is false, keeping the diagnostic body intact. Consider whether Redis failure should also fail the check — it degrades real-time delivery but leaves core HTTP working, so it may warrant a separate readiness endpoint rather than the same one.

G-02 — Staging cannot be provisioned through CI

`rds.tf` was added to the shared configuration, but the `ocar-prod-gha-staging` role's policy in `github-oidc.tf` grants only `ec2`, `elasticloadbalancing`, `autoscaling`, `elasticache` and `acm`. There is no `rds:*` statement, and no `secretsmanager:*` — which `manage_master_user_password` also requires.

A staging apply will proceed until it reaches the database resources and then fail on `AccessDenied`. Because this is the environment the load test depends on, this blocks `OPS-09`, which in turn blocks the load test, which in turn blocks the capacity validation the entire scale-readiness report defers to.

G-05 / G-06 — Missing approval gates

Three protections referenced in the workflows do not actually exist:

DECLARED	REALITY	CONSEQUENCE
<code>environment: production</code> in <code>deploy.yml</code>	Environment never created	Deploys to production run with no human approval pause.
<code>environment: staging</code> in <code>staging-infra.yml</code>	Environment never created	<code>terraform destroy -auto-approve</code> can be dispatched by anyone with workflow-run access.
Branch protection on <code>main</code>	Not configured	A direct push bypasses review. CI still gates <i>deploy</i> , but nothing gates <i>merge</i> .

All three require repository admin rights, which the account used to build this pipeline does not hold (`gh api repos/:owner/:repo --jq '.permissions'` returns `admin: false`). The exact `gh api` commands to close each are documented in `CLAUDE.md` under "Pending Ops Actions". **This is the single highest-value item on this list for whoever holds admin rights** — it is minutes of work and closes three distinct paths to unreviewed production change.

G-14 — Admin TOTP enforcement is bypassed

`api/src/middleware/auth.middleware.ts` line 125:

```
// TODO(SHIP-BLOCKER): hardcoded bypass, see TOTP_DEV_NOTE.md – remove before shipping  
const mustEnrollTotp = false && MANDATORY_TOTP_ROLES.has(adminRole) && !  
totpEnabled
```

The leading `false &&` short-circuits the entire expression, so `super_admin` and `finance_admin` accounts are never required to enrol in two-factor authentication despite the surrounding logic being complete and correct. The enrolment endpoints work; the enforcement is simply switched off. The comment marks it as a ship blocker. It is flagged here because these are the roles with the broadest access in the system.

G-13 — Test coverage

Five test files exist in the API — four covering ride assignment and broadcast, one covering maintenance middleware. There are none for payments, wallet, settlements, authentication, OTP, safety or ride chat. `vitest` and `@vitest/coverage-v8` are installed and `pnpm test:coverage` is defined, but CI runs `pnpm test` with no threshold.

This is worth stating plainly: the delivery pipeline is well-built and the gates are structurally sound, but a gate is only as strong as the checks behind it. Payments and authentication are the two areas where a regression is most expensive and where there is currently no automated protection at all.

17.3 Notes on selected medium and low items

REF	DETAIL
G-04	The comment above <code>aws_iam_role_policy_attachment.ssm_session_manager_debug</code> in <code>iam.tf</code> describes it as temporary debugging access to be removed once boot is confirmed working. That is now wrong and dangerous — <code>deploy.yml</code> routes migrations through <code>ssm:SendCommand</code> , which requires instances to be registered SSM managed instances. Acting on the comment would break every deploy. The attachment should stay; the comment must be corrected.
G-07	<code>min_size</code> , <code>desired_capacity</code> and <code>max_size</code> are literals in <code>asg.tf</code> , not variables. Staging therefore provisions the same 2/2/4 fleet as production. For a load-test environment that is arguably correct — the numbers must be comparable — but it removes the option of a cheaper single-instance staging for functional testing.
G-08	The VPC, subnets, internet gateway, route table and the two core security groups are named <code>\${var.project_name}-*</code> without the environment segment, unlike every other resource. Both environments would produce identically-named resources. They live in separate VPCs and separate state, so nothing collides functionally — but console navigation with both environments running is needlessly confusing.
G-17	cAdvisor and the <code>ocar_api</code> resource limits are committed to <code>docker-compose.prod.yml</code> , which reaches instances through an SSM parameter sourced from that file at <code>terraform apply</code> time. Until an apply runs and the fleet rolls, running instances do not have them. Applying also requires re-importing <code>infra/grafana/dashboards/ocar-overview.json</code> for the per-container row to render.
G-18	No <code>deregistration_delay</code> is set on the target group, so it uses the AWS default of 300 seconds. For long-lived WebSocket connections this is generous rather than insufficient — the concern with an unset value is usually the opposite. Worth setting explicitly so the value is visible in code rather than implied.

17.4 Suggested order of work

1. **G-06**, **G-05** — create the GitHub Environments and branch protection. Minutes of work, closes three unreviewed-change paths. Requires admin.
2. **G-01** — make `/health` return 503 on database failure. A few lines, and it is what makes the ASG's self-healing actually cover a real failure mode.
3. **G-14** — remove the TOTP bypass, or record an explicit decision to defer it with a date.
4. **G-02**, **G-03** — add the missing IAM permissions. Unblocks staging, which unblocks the load test.
5. **Load test** — resolves **G-09** and **G-16** with data rather than estimates, and validates the capacity claims in the readiness report.
6. **G-13** — payments and auth test coverage. The largest effort here, and the one that most changes the pipeline's real value.
7. **G-11**, **G-04** — reconcile the documentation with reality.

Appendices

A — Where everything lives

<code>.github/workflows/</code>	
<code>ci.yml</code>	7 parallel checks; gates everything downstream
<code>deploy.yml</code>	build → migrate → refresh → verify → rollback
<code>terraform-plan.yml</code>	posts a plan on any PR touching <code>infra/**</code>
<code>staging-infra.yml</code>	manual apply/destroy for staging
<code>infra/terraform/</code>	
<code>providers.tf</code>	versions; deliberately empty S3 backend block
<code>variables.tf</code>	every tunable input and its default
<code>prod.backend.hcl</code>	state key: <code>prod/terraform.tfstate</code>
<code>staging.backend.hcl</code>	state key: <code>staging/terraform.tfstate</code>
<code>staging.tfvars</code>	the only two values staging overrides
<code>vpc.tf</code>	VPC, 2 public subnets, IGW, route table
<code>security-groups.tf</code>	alb-sg and ec2-sg
<code>alb.tf</code>	ALB, target group, listeners, maintenance rule
<code>asg.tf</code>	Auto Scaling Group + target-tracking policy
<code>launch-template.tf</code>	AMI lookup, instance type, <code>user_data</code>
<code>templates/user_data.sh.tpl</code>	the entire instance boot sequence
<code>iam.tf</code>	EC2 instance role and SSM parameter names
<code>github-oidc.tf</code>	the three GitHub Actions roles
<code>rds.tf</code>	Postgres instance and parameter group
<code>elasticache.tf</code>	Valkey replication group and parameter group
<code>ssm-observability.tf</code>	repo files published as SSM parameters
<code>bootstrap/</code>	state bucket, KMS key, OIDC provider (own state)
<code>observability/</code>	Grafana alerts, SLOs, synthetics (own state)
<code>api/</code>	
<code>Dockerfile</code>	4-stage build; runs as non-root
<code>entrypoint.sh</code>	migrate, then start the server
<code>src/app.ts</code>	middleware chain, <code>/health</code> , <code>/metrics</code>
<code>src/server.ts</code>	worker startup and graceful shutdown
<code>src/websocket/socket.server.ts</code>	Socket.io, Redis adapter, rooms
<code>src/db/migrations/</code>	forward-only, numerically ordered
<code>docker-compose.prod.yml</code>	the five containers each instance runs
<code>infra/alloy/config.alloy</code>	log, metric and trace pipelines
<code>infra/grafana/dashboards/</code>	<code>ocar-overview.json</code>
<code>load-tests/</code>	k6 suite, seeding and verification

docs/OPS_RUNBOOK.md
CLAUDE.md

the short-form operational reference
living project state and pending actions

B — Command quick reference

PURPOSE	COMMAND
Who am I / which account	<code>aws sts get-caller-identity</code>
Find the ASG	<code>aws autoscaling describe-auto-scaling-groups --query 'AutoScalingGroups[].AutoScalingGroupName'</code>
Fleet state	<code>aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names <ASG></code>
Target health	<code>aws elbv2 describe-target-health --target-group-arn <TG_ARN></code>
Currently deployed version	<code>aws ssm get-parameter --name /ocar/prod/image-tag</code>
Start a fleet roll	<code>aws autoscaling start-instance-refresh --auto-scaling-group-name <ASG> --preferences '{"MinHealthyPercentage":100,"MaxHealthyPercentage":100}'</code>
Refresh progress	<code>aws autoscaling describe-instance-refreshes --auto-scaling-group-name <ASG> --max-records 1</code>
Cancel a refresh	<code>aws autoscaling cancel-instance-refresh --auto-scaling-group-name <ASG></code>
Shell into an instance	<code>aws ssm start-session --target <INSTANCE_ID></code>
Health / smoke	<code>curl -s https://ocar-api.clienttesting.in/health jq</code>
Terraform, production	<code>pnpm infra:prod:init :plan :apply</code>
Terraform, staging	<code>pnpm infra:staging:init :plan :apply :destroy</code>

C — Glossary

TERM	MEANING IN THIS SYSTEM
Instance refresh	The EC2 Auto Scaling mechanism that replaces instances in a group according to a policy. Used here for both deploys and infrastructure changes.
Launch before terminating	Replacement method triggered by <code>MinHealthyPercentage = 100</code> : a new instance must be healthy before the old one is terminated.
Target tracking	Scaling policy type where AWS maintains a metric at a target value and manages the alarms itself.
<code>ALBRequestCountPerTarget</code>	Requests per healthy target per one-minute period. Target value 1000 $\approx$ 16.7 req/sec per instance.
Deregistration delay	How long the load balancer lets in-flight work finish on a target being removed. 300 s here.
Health check grace period	How long a newly launched instance is exempt from health evaluation. 240 s here.
OIDC federation	Short-lived AWS credentials issued against a signed GitHub identity token, replacing stored access keys.
Drift	Divergence between real infrastructure and Terraform state. Created deliberately by <code>OPS-04</code> ; otherwise a problem.
Forward-only migration	Schema changes are never reverted. A bad migration is corrected by a new one.

D — Verification record

Every claim in this document was checked against the repository at `main` @ 4a07a95 on 20 August 2026. The following were read in full rather than sampled: all four workflow files; all sixteen Terraform files in `infra/terraform/` ; `user_data.sh.tpl` ; `docker-compose.prod.yml` ; `api/Dockerfile` ; `api/entrypoint.sh` ; `api/src/server.ts` ; `api/src/websocket/socket.server.ts` ; `api/src/middleware/auth.middleware.ts` ; `infra/alloy/config.alloy` ; `docs/OPS_RUNBOOK.md` ; and the health-check portion of `api/src/app.ts` .

Two behaviours that depend on AWS semantics rather than on this repository's code were verified against AWS's current published documentation rather than inferred: the effect of `MinHealthyPercentage` and `MaxHealthyPercentage` both set to 100, and the default target-group deregistration delay of 300 seconds.

Where this document and the codebase disagree, the codebase is correct and this document is out of date — please raise it. Where this document and

docs/architecture/2026-08-04-production-scale-readiness-report.md disagree,
see gap G-11 ; that report predates several of the decisions described here.